

Kamen škarje papir

Igra za Unity z Leap senzorjem

Ana Gaja Boc

13. oktober 2022

1 Uvod

Izdelali smo igro v Unity-u, kjer lahko igralec proti računalniku igra kamen škarje papir. [Leap Motion](#) senzor je uporabljen za zaznavanje pozicije roke v prostoru za interakcijo z uporabniškim vmesnikom in pa kote posameznih členkov v prstih, ki jih potrebujemo za prepoznavanje kretenj.

2 Kako namestiti in igrati igro

2.1 Kako namestiti

Celotna koda in zadnja verzija je dostopna na [githubu](#). Igra deluje na windows operacijskem sistemu, predhodno pa je potrebno naložiti [Leap Hand Tracking Software](#).

2.2 Kako igrati

Ko prižgemo igro lahko vidimo sceno, kakršna je na Sliki 1. S pomočjo senzorja lahko premikamo roko, ki je na desni strani zaslona. Paziti moramo, da je senzor pravilno obrnjen (s priključkom na levi strani), da zaznavanje roke deluje. Za začetek igre, se moramo s kazalcem dotakniti napisa *Igraj!*, kar sproži odštevanje. Ko se le to konča, računalnik izbere naključno potezo, jo primerja s potezo, ki jo je izbral nasprotnik ter razglasi rezultat. Če želimo igrati še eno igro, potem se s kazalcem ponovno dotaknemo gumba za začetek igre.

Igra ima tudi možnost goljufanja - računalnik prepozna igralčevo potezo in vedno izbere tako potezo, da igralec izgubi, sam pa zmaga. To možnost se lahko vključi in izključi s klikom z miško na gumb, ki se nahaja v zgornjem levem kotu zaslona. Gumb je skrit in se prikaže, ko se nad njim nahaja miška. Da je način goljufanja vključen vemo tudi po tem, da se barva ognja na sredini zaslona spremeni iz bele v modro.

Za izhod iz igre lahko pritisnemo na tipko ESC na tipkovnici ali pa z miško kliknemo na križec v zgornjem desnem kotu zaslona, ki se prikaže, če se nad njim nahaja miška.

3 Metodologija in implementacija

3.1 Logika poteka igre

Ob pritisku na gumb za začetek igre se shrani čas. Glede na ta čas se nato vsako sekundo izpiše preostali čas odštevanja. Prav tako se sproži animacija računalniško vodene roke, da se ta premika navzgor in navzdol, tako kot je običajno za to igro.

Ko se konča odštevanje, se pokliče algoritem za prepoznavanje poteze igralca. Glede na izbran način igranja, pravično ali goljufivo, nato računalnik ali naključno ali pa tako da mu zagotovi zmago izbere potezo. Za izbrano potezo se nato sproži animacija kretnje. Na zaslonu se tudi izpiše rezultat in po nekaj časa se ponovno aktivira gumb za igranje še ene igre.

Za popestritev igre so tudi dodane animacije, ki se sprožijo, če nekaj časa nihče ne igra igre.



Slika 1: Zajem zaslona igre: leva roka prikazuje kretnje računalnika, desna roka pa s pomočjo senzorja zazna kretnje igralca.

3.2 Prepoznavanje potez

Za prepoznavanje, katero potezo je izbral igralec je uporabljen KNN (k-najbližjih sosedov) algoritem. Predhodno smo shranili štiri kote za sklepe v vsakem od štirih prstov (vsi razen palca). Za vsako izmed treh možnih kretenj (kamen, škarje ali papir) smo zajeli 300 meritev. Naš algoritem ob koncu odštevanja tako poišče k število kretenj, ki so najbližje trenutni igralčevi.

Za metriko podobnosti se uporablja kot med rotacijama obeh kretenj. Za vsak členek v vsakem prstu igralca se izračuna, za kolikšen kot je le ta oddaljen od vnaprej posnetih kotov v členkih za vsako izmed treh potez. Oddaljenost se nato izračuna kot seštevek razlik v vseh členkih, z odstranjenjo največjo in najmanjšo meritvijo, za večjo robustnost.

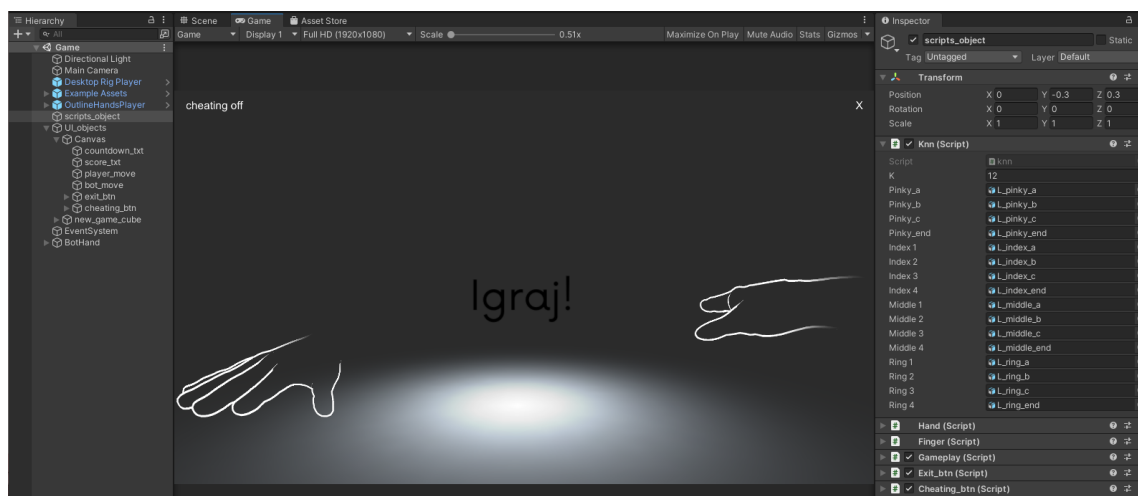
3.3 Postavitev Unity scene

Postavitev scene je vidna na sliki zajema zaslona 2. V sceni se nahajajo nujno potrebni elementi, kot so luč, kamera in tla. Poleg teh se v *UI_Objects* objektu nahajajo gumbi za igranje, izhod in goljufanje ter vsi teksti, ki se med igro izpisujejo na zaslonu. Gumb za začetek igre je pravzaprav kocka v prostoru in se za razliko od ostalih ne nahaja na objektu *Canvas*, saj mora biti 3D objekt v prostoru igre, da lahko z njim manipuliramo preko senzorja.

V sceni se nahajata tudi obe roki. Prva, s katero igra računalnik je pod imenom *BotHand*. Druga roka, katera je povezana na senzor se imenuje *OutlineHandsPlayer*. Ta objekt vsebuje tudi onemogočeno levo roko, tako da v igri sledimo le desni roki. Za pravilno delovanje in povezavo s senzorjem, potrebujemo tudi objekt *DesktopRigPlayer*. Vsa programska koda potrebna za delovanje sledenja roki, se nahaja na teh dveh objektih.

Vsa ostala programska koda potrebna za delovanje igre se nahaja na objektu *scripts_object*. Razreda *Hand* in *Finger* sta definirana za lažje delo s koti v roki. *Hand* vsebuje podatke o vseh prstih na roki in objekt *Finger* vsebuje podatke o vseh kotih v prstu. Razreda *Exit_btn* in *Cheating_btn* sta potrebna za delovanje gumba ki vključi/izključi goljufanje in pa gumba za izhod iz igre. Programska koda, ki narekuje potek igre, kot je opisano v poglavju o logiki poteka igre, se nahaja v razredu *Gameplay* in kliče razred *Knn*, v katerem se nahaja algoritem k-najbližjih sosedov in funkcija, ki računa razdalje med danima rokama.

Podatki potrebni za delovanje Knn se nahajajo v projektu v mapi *StreamingAssets*. Posneti so bili s pomočjo kode v razredu *RecordData*, ki se nahaja v drugi sceni v projektu, namenjeni snemanju in vizualizaciji posnetih podatkov. Ta druga scena se ne nahaja v končni verziji igre.



Slika 2: Zajem zaslona scene v Unity-u: na levi je hierarhija elementov v sceni, na desni pa so vsa programska koda potrebna za delovanje igre.

4 Rezultati

Prepoznavanje kretenj deluje za papir in kamen s 100% natančnostjo. Do napak občasno prihaja pri kretnji škarij, ko senzor napačno zazna sredinec ali kazalec kot pokrčen namesto iztegnjen ter nato kretnjo klasificira kot kamen. Do tega pride, kadar igramo pravokotno na senzor in sta prsta vertikalno poravnana, ter tako sredinec zakriva kazalec pred senzorjem. Problem lahko rešimo z igranjem vzporedno s senzorjem. Včasih tudi senzor ne zazna roke, četudi je ta v njegovem vidnem polju.

5 Zaključek

Izdelali smo igro kamen škarje papir, kjer lahko igralec s pomočjo Leap Motion senzorja igra proti računalniku. Senzor zazna igralčevo kretnjo in jo primerja proti kretnji, ki jo je izbral računalnik in proglasi zmagovalca. Igra deluje hitro in dovolj natančno za zabavne namene za katere je bila ustvarjena.